

Exercise Sheet 1

No Fear of Numbers: Introduction to Quantitative Data Analysis in R

Dr. Susanne de Vogel & Dr. Maryam Movahedifar, Data Science Center, University of Bremen

11 March 2026

Research Question: Is there a Life beyond the PhD?

In this workshop, we will work with one coherent example throughout all four parts. We focus on two outcomes of interest:

- `bpsy01`: overall life satisfaction (10 point scale)
- `blcd06`: Children (yes/no)

We will examine how these outcomes are related to different aspects of doctoral researchers' lives and backgrounds:

PhD and Work conditions

- `adbi01`: Status of the doctorate
- `adbi15`: Discipline
- `bcd17`: Emotional Support during PhD
- `bwd12`: Perceived scientific pressure
- `bemp81`: Monthly gross income

Attitudes and Well-Being

- `blcd12`: Satisfaction with work-life balance
- `apar15b`: Relationship with parents
- `bpsy05`: Self-Efficacy

Demographics

- `adem01`: Gender
- `adem02`: Age in years
- `apar10`: Highest vocational degree of parents
- `adem03`: Country of birth
- `blcd01`: Relationship status

Exploring datasets

In this exercise, we will use R to explore the data frame. The goal is to learn different ways to *examine the data frame and its variables* as a *preparation for analysis*. You will learn how to

- use different functions to inspect the data frame's structure
- examine variable types and classes
- check labels and missing values

0 Set-up: Installing and loading packages, loading the data frame

Okaaaaaaay, let's go!

0.1. Install tidyverse and haven if not already installed.

When using the packages for the *first time*, you could simply install it with `install.packages()`, e.g. `install.packages("tidyverse")`.

Alternatively, you can use a *conditional check* like

```
if (!requireNamespace("tidyverse", quietly = TRUE)) install.packages("tidyverse")
```

`requireNamespace("tidyverse", quietly = TRUE)` returns TRUE if the package is available. The `!` means “not”, so the condition becomes TRUE only when the package is missing. In that case, `install.packages("tidyverse")` is executed. This checks automatically whether the package is already installed.

To do so, start your script with the following code:

```
# Install tidyverse
if (!requireNamespace("tidyverse", quietly = TRUE)) # quietly = TRUE means that R runs the function
  ↪ silently
  install.packages("tidyverse")

# Install haven
if (!requireNamespace("haven", quietly = TRUE))
  install.packages("haven")
```

0.2. Use the `library()` function to load the packages *tidyverse* and *haven*.

Hint: Use the name of the package, but *without* quotation marks.

0.3. Define the *Exercises* folder as your working directory with the function `setwd("path")`.

Hint: Note that R uses a slash/ instead of a backslash .

0.4. Load the SPSS file “01_qa_exploring_data.sav” as a data frame using the `read_sav("dataset file name")` function and call this data frame `mydata1`.

Exercise 1: Exploring the structure of your data frame

To check whether the data are in a tidy structure, **tidyverse** provides a range of functions that allow us to explore the data frame and its variables.

1.1. On the right side, in the Environment pane, the data frame `mydata1` should now appear under *Data*. What kind of information about the data frame do you find here?

1.2. Use the function `glimpse(mydata1)` to take a first look at your data frame. What kind of information do you get from the output?

1.3. Execute the functions `head(mydata1)` and `tail(mydata1)`. What is the difference to `glimpse()`? Which information do you get?

1.4. Inspect the data frame in the RStudio Data Viewer using the function `View(mydata1)`. What can you see here?

Be careful: R is case sensitive, so use V with a capital letter!

Exercise 2: Inspecting categorical variables

To understand what kind of variables we are dealing with, we now inspect a few example variables in more detail.

Your task is to find out which type and class of variable we have, which labels are available, whether there are any missings (NAs).

To call not a whole data frame, but a *specific variable* in that data frame, write `mydata1$adem01`. You can read this as: “the variable `adem01` from the data frame `mydata1`”:

- `mydata1` = name of the data frame (the table)
- `$` = “take this specific column from the table”
- `adem01` = name of the variable / column

2.1. Use the functions `typeof()` and `class()` to inspect the variable `adem01`. What type and class is it in R?

2.2. Use `attr()` with the options “label” and “labels” to check whether a variable label and value labels are stored for `adem01`.

Code it like this:

```
attr(mydata1$adem01, "label")      # variable label
attr(mydata1$adem01, "labels")     # value labels
```

2.3. Create a simple frequency table for `adem01` using `count()`.

Use the pipe operator `%>%` to first pass the whole data frame into `count()`, e.g. `mydata1 %>% count(adem01)`, because `count()` needs a data frame as input and cannot be applied directly to a single variable like `mydata1$adem01`.

Code it like this:

```
# Take the dataset mydata1 and then count the occurrences of each value of adem01.
mydata1 %>%
  count(adem01)
```

2.4. Check whether there are any missing values for `adem01` using the `sum(is.na())` function.

Exercise 3: Inspecting numeric variables

3.1. Use the functions `typeof()` and `class()` to check the type and class of the variable `adem02`.

3.2. Use `attr()` to check the variable and value labels. Anything noticeable?

3.3. Use `summary()` and `count()` (remember to write `count()` as a pipe) to get a quick overview of the range of ages.